

A Succinct and Approximate Greedy Algorithm for the Minimum Set Cover Problem

Jorge Delgado, Héctor Ferrada and Cristóbal A. Navarro

Instituto de Informática, Universidad Austral de Chile, Chile

Abstract

The Minimum Set Cover Problem (MSCP) is a combinatorial optimization problem belonging to the NP-Hard class in computer science. For this reason, there is no algorithm that in the worst case ensures finding an optimal solution in polynomial-time. For a given universe $\mathcal{X}$, the popular greedy heuristic, called **Greedy-SetCover**, is the main theoretical contribution to obtain an approximate solution for the MSCP in polynomial-time, offering an optimal approximate ratio of $(\ln |\mathcal{X}| + 1)$. In this article, we propose an approximate algorithm for MSCP within a succinct representation of the input dataset, whose empirical performance improves **Greedy-SetCover** both in quality and execution time, while offering the same optimal approximation ratio for the problem. Our experiments show that the proposed algorithm is magnitudes of times faster than the aforementioned greedy one, obtaining on average a cardinality much closer to the optimal solution. Furthermore, because we work on a succinct representation that allows us to compute operations between sets using bitwise operators, we can process much larger datasets than state-of-the-art solutions. As a result, our proposal is also a suitable alternative for processing large datasets as required by the current Big Data era.

Keywords: Combinatorial optimization, Set cover, NP-Hard, Greedy algorithms, Approximate algorithms

1. Introduction

A *set-covering* for a given universe $\mathcal{X}$ is a group of subsets of $\mathcal{X}$ whose union includes all the elements of $\mathcal{X}$. Since there can be many different combinations of sets whose union gives the same universe, in some circumstances

12 it is desired to select only the best combination, or optimal, among all the
 13 available sets. When we define the optimum as the smallest number of sets to
 14 cover $\mathcal{X}$ and we require to determine what this optimal solution is, then we
 15 are talking about the Minimum Set Cover Problem (MSCP). The MSCP is
 16 an NP-hard optimization problem [21, 28] that we formally define as follows.
 17 Given an instance $(\mathcal{X}, \mathcal{F})$ as input dataset, where $\mathcal{F} = \{S_1, \dots, S_m\}$ is the
 18 family of subsets and $\mathcal{X}$ is the universe formed by the union of all sets in $\mathcal{F}$,
 19 the problem consists in to find a set-covering $\mathcal{C} \subseteq \mathcal{F}$ of $\mathcal{X}$; i.e., $\mathcal{X} = \bigcup_{S \in \mathcal{C}} S$ of
 20 minimum size. According to the definition it is possible that there is more
 21 than one solution for MSCP, in which case all of them are called optimal so-
 22 lutions for the given instance $(\mathcal{X}, \mathcal{F})$, and to solve MSCP, it is enough to find
 23 any of them. The Figure 1 shows an universe of twelve elements distributed
 24 in seven subsets $S_i \in \mathcal{F}, 1 \leq i \leq |\mathcal{F}|$ which are the input instance $(\mathcal{X}, \mathcal{F})$
 25 for the problem, and for this input dataset the unique optimal solution is
 26 $\mathcal{C}^* = \{S_1, S_4, S_5\}$.

27

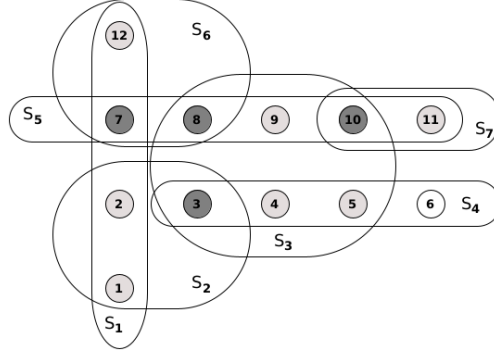


Figure 1: Illustration of an instance $(\mathcal{X}, \mathcal{F})$ of the MSCP, with $|\mathcal{X}| = 12$ elements and $\mathcal{F} = \{S_1, \dots, S_7\}$, where $S_1 = \{1, 2, 7, 12\}, S_2 = \{1, 2, 3\}, S_3 = \{3, 4, 5, 8, 9, 10\}, S_4 = \{3, 4, 5, 6\}, S_5 = \{7, 8, 9, 10, 11\}, S_6 = \{7, 8, 12\}, S_7 = \{10, 11\}$. The optimal solution for this example is the minimum set-covering $\mathcal{C}^* = \{S_1, S_4, S_5\}$.

28 The MSCP is widely used in many optimization problems that require
 29 resources to be allocated (see, e.g., [1, 14, 24, 43]). As a small example
 30 in combinatorics, within a multidisciplinary research project, we may need
 31 to select the minimum number of researchers to cover all the research ar-
 32 eas and skills $\mathcal{X}$ necessary to carry out the project. Thus, a solution for
 33 MSCP it is enough. MSCP also appears naturally in tasks of coordination

34 and distribution of finite resources within the industry. For instance, within
 35 industrial planning, in order to avoid overlaps and/or reduce operating costs,
 36 an optimal solution may be required to allocate worker shifts. In the same
 37 line of planning, the Transit Route Network Design Problem (TrNDP) can
 38 be derived from the MSCP [36] and use this solution to establish transport
 39 routes of a company. In another application, some model-based view plan-
 40 ning methods also require a set-covering to solve the View Planning Problem
 41 (VPP), which involves determining the minimum number of views that must
 42 be used to reconstruct the part in 3D [39]. In industrial robot coordination,
 43 a solution for MSCP also is used to minimize synchronization points required
 44 by its algorithms [42]. A set-covering also appears as an optimization prob-
 45 lem to make an efficient distribution of tasks in an *Internet of Things* (IoT)
 46 network within the context of limited energy [13]. In *Crew scheduling*, the
 47 problem of assigning a group of workers (a *crew*) to a set of tasks can be
 48 formulated from the MSCP (see, e.g., [6, 16, 41]). Contardo et al., [20], pro-
 49 pose a hybrid algorithm that iterates between computing a set-covering and
 50 constructing a Voronoi diagram to solve MSCP in geometric contexts. The
 51 MSCP has many other applications in different areas of research on optimiza-
 52 tion problems, such as industrial applications [17], information retrieval [44],
 53 in graph theory to determine a transversal in hypergraphs [18], in satellite
 54 systems for traffic allocation [40], among others.

55
 56 In the worst case, any current method that determines the optimal com-
 57 bination of sets from $\mathcal{F}$, requires at least exponential time in the size of the
 58 family of sets, $|\mathcal{F}| = m$. We can homologate all the possible combinations
 59 from $\mathcal{F}$ of an exhaustive algorithm as if it were a binary counter of m bits,
 60 starting from 1 to 2^m . The binary counter determines each possible sets
 61 combination, where if the i th bit is one in a combination ρ , $1 \leq i \leq m$,
 62 then the set S_i is included in ρ . Thus, we need to evaluate each one of the
 63 2^m states of the counter, checking if the combinations correspond to a true
 64 set-covering for $\mathcal{X}$, and remember the configuration with the fewest sets. In
 65 order to check if a group of sets includes all the elements of $\mathcal{X}$, a validation
 66 needs to be performed on all $n = |\mathcal{X}|$ elements, which requires at least linear
 67 time in the number of elements. In conclusion, the worst-case time of opti-
 68 mal algorithms based on this exhaustive search to solve the MSCP cannot be
 69 less than $n \cdot 2^m$. Although for specific inputs perhaps it is possible to prune
 70 the search space and reduce this time, but it will never be possible in the
 71 worst-case using an exhaustive search strategy, which is the only guaranteed

method today that always finds the optimal one for the MSCP.

In this sense, there are many heuristics to improve, in some cases, the execution time of the exhaustive search that obtains the optimal solution to the problem. For example, we can narrow the search space by determining the smallest number of sets that any optimal solution must contain, as follows. If we determine the smallest integer k such that the sum of the sizes of the k largest subsets of $\mathcal{F}$ is greater than or equal to $|\mathcal{X}|$, then we know that any optimal solution must contain at least k subsets. In order to obtain this k value, we can sort in descending order on the sizes of subsets of the family $\mathcal{F}$ in $O(|\mathcal{F}| \lg |\mathcal{F}|)$ optimal time. Thus, having found this k , we start the search by evaluating any combination of k sets; if we do not find a set-covering for the universe, then we iterate again increasing k by one unit and repeating the search for this updated k value. When we find a set-covering for $\mathcal{X}$, we stop the iteration reporting the last k value as the size of the optimal solution.

As we have seen, the MSCP has a great impact on academia, scientific research and several industry fields. However, under the assumption that it is very unlikely to find a polynomial-time algorithm to obtain the optimal set-covering, because of its NP-completeness, great part of the effort of the scientific community is in the development of polynomial-time approximations for the problem. This means to offer faster solutions at the cost of a lower quality. Furthermore, we believe that in combination with efficient programming techniques, it is possible to build faster and/or smaller solutions to the problem, either to determine the optimal configuration or to design new approximate solutions.

In the body of this article, we use both big O notation and little o notation to specify asymptotic upper bounds. Given two asymptotically positive functions f and g , we say that f is $o(g)$ to indicate that f is always asymptotically less than g ; and we say that f is $O(g)$ if asymptotically f is at most as large as g [21].

The rest of this article is structured as follows; Section 2 details the related work in the field, the next Section 3 presents the new greedy solution for MSCP. Section 4 shows experimental results and finally Section 5 presents conclusions with future work.

2. Related Work

In 1972 Karp showed that the MSCP is NP-Hard [28], and therefore, very unlikely to be optimally solved in polynomial-time. Two decades later, Lund and Yannakakis based on the work of Karp, showed in [33] that it is not possible to obtain a polynomial-time approximate algorithm for solving the MSCP, with ratio better than $\frac{1}{4} \lg n$, unless NP has slightly superpolynomial-time deterministic algorithms, that is $O(n^{\text{poly} \log n})$ where $n = |\mathcal{X}|$ is the universe size and $\text{poly} \log n$ mean polynomial in $\log n$. Thus, $O(n^{\text{poly} \log n})$ means $O(n^{(\log n)^k})$ for some positive k constant, which is greater than polynomial-time. Therefore, every polynomial solution must have an approximate ratio $\geq \frac{1}{4} \log n$, showing that it is also difficult that a polynomial approximation method becomes $\leq \frac{1}{2} \log_2 n \simeq 0.72 \ln n$. A few years later, in 1998, Uriel Feige [22] adjusted the approximation ratio to $(1 - o(n)) \ln n$, but always considering that NP does not include slightly superpolynomial-time algorithms. Therefore, given the NP-completeness of the MSCP which indicates that every optimal solution has superpolynomial-time in the worst case, most of the current optimal strategies focus their success on the development of heuristics to prune the search space. The latter has the risk that for certain cases of the input instance it is not possible to do any pruning, or narrow the search only a little. For this reason, in the literature we find several optimal algorithms based on different pruning strategies for the search tree [3, 4, 7]. Among these types of algorithms, Lan et al. [31] find the optimal set-covering by applying a search strategy called meta-heuristics for random priority search, introducing randomness into the resolution of the problem. Other exhaustive search algorithms are mainly based on branch-and-bound and branch-and-cut methods [2, 8, 23].

Given the previous reasons, one of the greatest efforts in MSCP research is in the design of new approximate algorithm, thus avoiding the restrictions imposed by the NP-Hard classification of this problem. In this line, there are several proposals in the state-of-the-art that try to obtain faster solutions and desirably close to the optimum quality. In 1973 Johnson published his Greedy Algorithm [27] which determines an approximation for the MSCP in polynomial-time, labeled as Algorithm C1 in his article. Later, in 1979 Chvatal [19] investigated the work of Johnson to propose an approximate solution with very similar characteristics, which can be reduced to Johnson's results under certain conditions. We have implemented the Algorithm

145 1 described in Cormen et al. [21], which is frequently referred as the classic
146 greedy strategy to solve the MSCP in the literature. This method includes
147 an intuitive heuristic that consists of iteratively selecting the set that cov-
148 ers the greatest number of remaining uncovered elements in the universe,
149 subtracting the included items from the current universe and iterating un-
150 til all items are covered. Another interesting result appeared in [12], where
151 the authors identified two issues that arise when determining a set-covering
152 with metaheuristic approaches: solution infeasibility and set redundancy¹.
153 Accordingly, they proposed a solution on a 1-flip neighbourhood structure
154 to find a set-covering with metaheuristic by including a penalty approach to
155 obtain an approximate solution to the problem. Lim et al. [32] also ana-
156 lyzed the main approach of Johnson’s greedy algorithm to solve the MSCP
157 by evaluating eager and lazy versions of the algorithm. Caprara et al. [15]
158 proposed an heuristic based on Lagrangian relaxation to find an approxima-
159 tion for the problem, whose implementation won an important competition
160 of that decade, called FASTER. Another paradigm of algorithm design in
161 the framework of solutions for MSCP is through randomized algorithms.
162 Peuzin-Jubert et al. [38] introduce an heuristic called randomized rounding
163 algorithm for MSCP, achieving an approximation ratio of $k(1 - (c/n)^{1/k})$,
164 where k and c are positive constants.

165
166 Gomes et al. [25] carried out an empirical study of approximation al-
167 gorithms for Set Cover Problems. They tested the classical greedy algo-
168 rithm to solve MSCP against four variants of it: Round [26], Dual-LP [26],
169 Primal-Dual [5], and Greedy Chvatal’s algorithm —which is extracted from
170 the Johnson’s algorithm. They conclude that the classical greedy approach
171 always perform better than the another three methods, both in time and
172 quality of the solution. Furthermore, after [25], although there are some
173 approximate proposals to solve the MSCP, these are mainly for the case of
174 weighted sets, where including a certain set as part of the solution has a
175 specific cost; but not for the case of the unweighted MSCP that is the focus
176 of study of this article. Therefore, we believe that the mere fact of offering
177 a solution to the approximate MCSP with better experimental results than

¹An incomplete set-covering is considered to be infeasible if one or more of the elements of the universe are uncovered (i.e., not really a solution). A set is considered to be redundant if all the elements covered by the set are also covered by other sets in the solution.

178 those obtained with Johnson’s algorithm, both in time and quality of the
 179 approximation, is in itself a great contribution in this area of study, not only
 180 in the maturity of the MSCP but also in the design of general solutions to
 181 NP-Hard problems.

182 2.1. The Greedy Set Cover Algorithm

183 The solution with major acceptance in the literature is the greedy ap-
 184 proximation introduced by Johnson in 1973 [27], which is also described in
 185 Cormen et al.[21]. This method is a polynomial-time algorithm whose heuristic
 186 selects at each iteration the most promising subset among those available.
 187 The key idea is to select the set $S \in \mathcal{F}$ that contains the largest number of
 188 elements that have not yet been included by any set of the partial solution
 189 that has been built up to that iteration. The process ends the iterations when
 190 all the elements of the universe have been considered by at least one of the
 191 selected sets. This strategy ensures finding a solution $\mathcal{C}$ whose approximation
 192 relation is $\frac{|\mathcal{C}|}{|\mathcal{C}^*|} \leq \ln |\mathcal{X}| + 1$, where $\mathcal{C}^*$ is an optimal solution for the prob-
 193 lem. The pseudocode described in Algorithm 1 corresponds to the greedy Set
 194 Cover algorithm detailed in Cormen et al. [21] with $O(\min\{|\mathcal{X}|, |\mathcal{F}|\} \cdot |\mathcal{X}| \cdot |\mathcal{F}|)$
 195 complexity time in the worst case, which is very similar to Johnson’s original
 196 method published in [27] and the greedy approach of Chvatal in [19]. The
 197 polynomial theoretical time is explained by the loop of lines 4–8, where it
 198 is necessary to determine the set S that maximizes $|\mathcal{U} \cap S|$. To do that, in
 199 each of the $\min\{|\mathcal{X}|, |\mathcal{F}|\}$ iterations, we need to examine up to $|\mathcal{F}|$ subsets
 200 of size $|\mathcal{X}|$ in the worst case, obtaining an asymptotic time of $O(|\mathcal{X}| \cdot |\mathcal{F}|)$ in
 201 the worst case.

202 The correctness of Algorithm 1 is ensured by the loop condition, which does
 203 not allow to finish the process until all the elements of the universe have
 204 been considered in the solution. The quality of the greedy algorithm has an
 205 approximation ratio of $O(\ln |\mathcal{X}|)$ [21], which is theoretically optimal consid-
 206 ering the conditions for the problem discussed in the Introduction (Section
 207 1).

208 3. An Approximate Greedy Set Cover Algorithm in a Succinct 209 Data Representation

210 The classic greedy strategy described above achieves its time and quality
 211 by selecting the set with the largest number of uncovered elements by the

Algorithm 1 GREEDY-SETCOVER algorithm in [21] to compute a set-covering for the universe $\mathcal{X}$ and the family of subsets $\mathcal{F}$

Require: The universe $\mathcal{X}$ and the family of subsets $\mathcal{F}$ whose union is $\mathcal{X}$

```

1: procedure GREEDY-SETCOVER( $\mathcal{X}, \mathcal{F}$ )
2:    $\mathcal{U} = \mathcal{X}$ 
3:    $\mathcal{C} = \emptyset$ 
4:   while  $\mathcal{U} \neq \emptyset$  do  $\triangleright O(\min\{|\mathcal{X}|, |\mathcal{F}|\})$  time
5:     select  $S \in \mathcal{F}$  that maximizes  $|\mathcal{U} \cap S|$   $\triangleright O(|\mathcal{U}| \cdot |\mathcal{F}|)$  time
6:      $\mathcal{U} = \mathcal{U} - S$ 
7:      $\mathcal{C} = \mathcal{C} \cup S$ 
8:   end while
9:   return  $\mathcal{C}$ 
10: end procedure

```

212 partial solution. This polynomial-time Algorithm 1 is essentially the best
 213 theoretical research result to date regarding to approximate solutions for the
 214 set cover problem. However, the method can take too long when the size of
 215 the data collection is large. Mainly because iteratively performs exhaustive
 216 searches for the best set among all available sets in $\mathcal{F}$, validating in the worst
 217 case up to $|\mathcal{F}|$ sets. Furthermore, the exaggerated generality of the algorithm
 218 invites the development of more adequate data structures to represent the
 219 sets; for instance, by using an environment that facilitates the computation
 220 of set operations such as Union and Intersection. These aspects have opened
 221 up a research opportunity that has given rise to this contribution.

222 Derived from Algorithm 1, we have designed a set cover approximated algo-
 223 rithm incorporating new heuristics to empirically improve the original greedy
 224 method. Thus achieving greater speedup and space savings, as well as guar-
 225 anteeing optimal asymptotic complexity in terms of approximation quality
 226 —considering that this problem is encapsulated by the NP-hard class. We
 227 highlight two key points in our proposal: *i*- the design of an efficient and
 228 lightweight data structure that allows calculations of operations between sets
 229 directly in RAM; and *ii*- a new heuristic to prune the space search of the
 230 best set in each iteration. The final part of our proposal is an implementation
 231 of the new algorithm that improves both the execution time of the original
 232 method and the quality of the approximation.

233

234 When analyzing Algorithm 1, we can notice that the key step is in line
 235 5, when it selects the next set $S \in \mathcal{F}$ to be included in the solution. This
 236 selection has to check all the remaining sets of $\mathcal{F}$, obligating to execute an ex-

237 exhaustive search in $\mathcal{F}$, requiring $O(|\mathcal{X}| \cdot |\mathcal{F}|)$ time in the worst case. The time
 238 of this selection can become very significant for large sizes of the universe $\mathcal{X}$
 239 or the family of subsets $\mathcal{F}$. This is the most expensive task of each iteration
 240 in the algorithm, because most of these exhaustive searches are performed
 241 on a large subset of $\mathcal{F}$.

242
 243 Consider Figure 1, where the optimal solution is $\{S_1, S_4, S_5\}$. The clas-
 244 sic greedy algorithm starts by selecting the subset S_3 to build the solution
 245 $\mathcal{C} = \{S_3, S_1, S_4, S_7\}$, or including S_5 instead of S_7 in $\mathcal{C}$, whose size $|\mathcal{C}| = 4$.
 246 However, note that we can select S_4 as the first subset of the solution $\mathcal{C}$,
 247 because the element number 6, the rightmost in S_4 , is not included by any
 248 other set. In this way, we can reduce the search space if we focus only on
 249 covering the elements that are present in the smallest number of subsets.
 250 Also hoping that, with this way of selecting each set $S \in \mathcal{F}$, other elements
 251 of the universe will also be covered, resulting in a possible faster and more
 252 efficient algorithm than 1 and also with a better approximation ratio. Thus,
 253 we iteratively select the set containing the largest number of uncovered el-
 254 ements that are present in the smallest number of subsets in $\mathcal{F}$. Formally,
 255 if an item $x \in \mathcal{X}$ is included exactly in k sets, then we call x as an *ele-*
 256 *ment of grade k* . In the example of the Figure 1, the white element, the
 257 item with identifier 6, has grade $k = 1$; the light gray elements (identifiers
 258 1, 2, 4, 5, 9, 11, 12) have grade $k = 2$; and the dark gray elements (identifier
 259 3, 7, 8, 10) have grade $k = 3$. We will then use this notion in order to provide
 260 a new greedy algorithm to compute the set-covering for the instance $(\mathcal{X}, \mathcal{F})$.
 261 Below we formally detail our algorithm, which pseudocode is described in
 262 Algorithm 2.

263
 264 We first detail the function of each procedure included in the pseudocode
 265 of the algorithm.

- 266 • $\text{CREATEMAP}(\mathcal{X}, \mathcal{F})$, procedure that creates and returns a map data
 267 structure, called MAP, of pairs $(x_i, L_i[])$, where for each $x_i \in \mathcal{X}$, $L_i[]$ is
 268 the list of all subsets $S_i \in \mathcal{F}$ in which x_i is. It can be built in $O(|\mathcal{X}| \cdot |\mathcal{F}|)$
 269 time by simple iterative inspection.
- 270 • $\text{SORTUNIVERSE}(\text{MAP}, \mathcal{F})$, procedure that sorts the MAP structure of
 271 size $|\mathcal{X}|$ in increasing order with respect to the number of times that

Algorithm 2 SUCCINCT-SETCOVER algorithm to compute a set cover of the universe $\mathcal{X}$ and the family of subsets $\mathcal{F}$.

Require: The universe of elements $\mathcal{X}$ and the family of subsets $\mathcal{F}$ whose union is $\mathcal{X}$

```

1: procedure SUCCINCT-SETCOVER( $\mathcal{X}, \mathcal{F}$ )
2:   Let MAP be a map data structure of length  $|\mathcal{X}|$  to store, for each  $x_i \in \mathcal{X}$ , the lists
   of ID-subsets where  $x_i$  is.
3:   MAP  $\leftarrow$  CREATEMAP( $\mathcal{X}, \mathcal{F}$ )  $\triangleright O(|\mathcal{X}| \cdot |\mathcal{F}|)$  time
4:   SORTUNIVERSE(MAP,  $\mathcal{F}$ )  $\triangleright O(|\mathcal{X}| \log |\mathcal{X}|)$  time
5:    $\mathcal{C} = \emptyset$ 
6:    $\mathcal{U} = \mathcal{X}$ 
7:    $\langle ini, k \rangle \leftarrow$  NEXTGRADE(MAP, 0)
8:   while  $\mathcal{U} \neq \emptyset$  do  $\triangleright O(\min\{|\mathcal{X}|, |\mathcal{F}|\})$  time
9:      $end \leftarrow$  FINDRANGE(MAP,  $ini, k$ )  $\triangleright O(|\mathcal{X}|)$  time for all the calls
10:     $\ell = end - ini + 1$   $\triangleright$  numbers of elements of grade  $k$  to cover
11:    while  $\ell > 0$  do
12:       $\langle S, \delta \rangle \leftarrow$  FINDSUBSET(MAP,  $ini, end$ )  $\triangleright O(\frac{|\mathcal{X}|}{W} |\mathcal{F}|)$  time
13:       $\mathcal{U} = \mathcal{U} - S$   $\triangleright O(\frac{|\mathcal{X}|}{W})$  time
14:       $\mathcal{C} = \mathcal{C} \cup S$   $\triangleright O(\frac{|\mathcal{X}|}{W})$  time
15:       $\ell = \ell - \delta$ 
16:    end while
17:     $\langle ini, k \rangle \leftarrow$  NEXTGRADE(MAP,  $k$ )
18:  end while
19:  return  $\mathcal{C}$ 
20: end procedure

```

272 each $x_i \in \mathcal{X}$ appears in the subsets of $\mathcal{F}$. In this process, for each x_i ,
 273 with the same MAP, we obtain in constant time the length of each list
 274 of subsets that contain x_i . Thus, MAP is sorted in $O(|\mathcal{X}| \log |\mathcal{X}|)$ time.

275 • FINDRANGE(MAP, ini, k), procedure that determines and returns the
 276 right limit end of the range MAP[$ini \dots end$] of all elements $x \in \mathcal{X}$ of
 277 grade k . Thanks that MAP is sorted, determining each interval $I =$
 278 MAP[$ini \dots end$] can be done in linear time over the size $|I|$.

279 • FINDSUBSET(MAP, ini, end), procedure that determines and returns
 280 the pair $\langle S, \delta \rangle$, where $S \in \mathcal{F}$ is the subset that contains the major num-
 281 ber δ of minimal uncovered elements inside the range MAP[$ini \dots end$]
 282 of all elements with the same grade. If there are ties, then select
 283 the subset that maximizes $|S \cap \mathcal{U}|$ among all the lists MAP[x_i].*list*,
 284 $\forall x_i \in \text{MAP}[ini \dots end]$.

285 • NEXTGRADE(MAP, k), procedure that determines and returns the pair
 286 $\langle ini, k' \rangle$ for the next grade $k' > k$ with at least one uncovered element;
 287 ini is the index in MAP where this interval begins.

288 As input, we are given the instance $(\mathcal{X}, \mathcal{F})$, which are the universe of
 289 elements $\mathcal{X}$ and the family of subsets $\mathcal{F}$, whose union is $\mathcal{X}$. We also require
 290 to build a MAP data structure² to associate to each $x \in \mathcal{X}$ the list of all
 291 subsets $S \in \mathcal{F}$ where x is —created at the line 3 in the pseudocode. At line
 292 4, we sort the universe $\mathcal{X}$ in MAP (by the length of the list that contains each
 293 element) by calling to the procedure SORTUNIVERSE. On line 7, we search
 294 for the first degree $k \geq 1$ with at least one minimum element, the first call of
 295 this procedure always returns $ini = 1$. We build the solution $\mathcal{C}$ incrementally,
 296 which is initially an empty set —see line 5. We start iterating with $\mathcal{U} = \mathcal{X}$
 297 while $\mathcal{U}$ has elements, which means that the size of our solution $\mathcal{C}$ is lower
 298 than $|\mathcal{X}|$ —line 8. The process finds, at line 9, each range MAP[$ini \dots end$] of
 299 all elements $x \in \mathcal{X}$ of grade k , starting with $k = ini = 1$. These elements of
 300 grade k , are also called *Minimal Elements*, because we look for uncovered ele-
 301 ments of minimal grade k among all the remaining elements not yet included
 302 by the partial solution $\mathcal{C}$. Therefore, at the beginning, we start searching in

²This corresponds to a BST or a Red/Black Tree [21, 29], where each key has associate an additional data structure such as `std::map`, which is provided by the GNU C++ Standard Library and implements a Red/Black Tree.

303 MAP the interval of all elements that are included only by $k = 1$ subsets of
 304 $\mathcal{F}$; that is, in the first group of iterations (the inner loop at lines 11–16) the
 305 minimal elements are items of grade $k = 1$. Thus, in the next iterations of
 306 the while-cycle of lines 8–19 we search for the next interval of minimal items
 307 of grade $k = 2$. As in the MAP structure, the ranges of elements are grouped
 308 by their grade value, placing one after the other as the grade of the elements
 309 increases. Therefore, we now loop in the inner while-cycle, lines 11–16, while
 310 the range size ℓ be greater than 0. It means that there are uncovered ele-
 311 ments of grade k and, consequently, we must continue including subsets in
 312 the solution until we cover all the minimal elements of the range.

313

314 The correctness of the Algorithm 2 is given by the condition of the outer
 315 loop of line 8, which does not end as long as there is at least one uncovered
 316 element.

317

318 With the example of Figure 1, unlike the classic greedy algorithm that
 319 starts by searching exhaustively among all the subsets in $\mathcal{F}$ to select the
 320 subset S_3 , the first search of the SUCCINCT-SETCOVER algorithm is only
 321 among the sets that contains at least one element of minimal grade, since we
 322 start with $k = 1$, the only set that satisfies the condition is S_4 . The next
 323 selected set, for $k = 2$, is S_1 , which once included elements of grade 2 remain
 324 uncovered, so S_5 is chosen in the second iteration of the inner loop for $k = 2$.
 325 Note that in the example we never search for elements of grade $k = 3$ because
 326 these were implicitly included when we added the sets to cover the elements
 327 of grade $k = 1$ and $k = 2$.

328 3.1. Asymptotic Analysis of the SUCCINCT-SETCOVER Algorithm

329 We detail the asymptotic analysis of our SUCCINCT-SETCOVER Algo-
 330 rithm 2 for both, the time consumption and the quality of the solution.

331

332 To obtain a threshold of the time, let us break down the pseudocode
 333 of the Algorithm 2. At first we need to construct an ordered dictionary
 334 that relates element identifiers to subset identifiers. This order is given
 335 by the number of subsets in which each element is found. We can do
 336 this using any optimal sorting algorithm, requiring an asymptotic time of
 337 $O(|\mathcal{X}| \cdot |\mathcal{F}| + |\mathcal{X}| \log |\mathcal{X}|) = O(|\mathcal{X}| \cdot |\mathcal{F}|)$ —in our implementation we use
 338 the STD::MAP container and the INTROSORT algorithm [34] implementation
 339 available with STD::SORT(). Later, at line 9, all the calls for the procedure

340 FINDRANGE require $O(|\mathcal{X}|)$ time, which is achieved by taking advantage of
 341 the order in MAP, performing a simple scan. At line 12, each call of the pro-
 342 cedure FINDSUBSET(MAP, *ini*, *end*) requires a time of $len \cdot O(\frac{|\mathcal{X}|}{W}|\mathcal{F}|)$, with
 343 $len = end - ini + 1$, at the worst case with $len \leq \min\{|\mathcal{X}|, |\mathcal{F}|\}$. The expla-
 344 nation is given that we can solve any set operations between a pair of sets
 345 —such as UNION, INTERSECTION and DIFFERENCE— in linear time on the
 346 size $|\mathcal{X}|/W$, where W is the memory word size in the CPU architecture³.
 347 This is achieved with efficient computing techniques that are detailed in Sec-
 348 tion 3.2. Accordingly, we also take $O(|\mathcal{X}|/W)$ time for computing $\mathcal{U} - S$ and
 349 $\mathcal{C} \cup S$, at lines 13—14. Altogether, the total run time of our algorithm is
 350 $O(\min\{|\mathcal{X}|, |\mathcal{F}|\} \frac{|\mathcal{X}| \cdot |\mathcal{F}|}{W})$ at the worst case, which is clearly polynomial.

351
 352 Let us now continue with the detail of the approximation relation of our
 353 solution, which we will proof that it cannot be greater than $|\mathcal{C}^*|(\ln |\mathcal{X}| + 1)$
 354 in the worst case, where $\mathcal{C}^*$ is an optimal solution for the instance. Let S_i
 355 denote the i th subset selected by our SUCCINCT-SETCOVER Algorithm 2
 356 and k_i the current grade of minimal items at the i th iteration respect to
 357 the inner loop (lines 11—16). Thus, the i th iteration corresponds when the
 358 procedure FINDSUBSET(...) selects the best subset $S_i \in \mathcal{F}$ from the range
 359 of grade k_i , which is the one that contains the largest number of uncovered
 360 minimal elements (i.e., uncovered elements of grade k_i). We also establish
 361 that each selected subset S_i contributes with a cost of 1 to $\mathcal{C}$, cost that is
 362 distributed by all the new minimal items $x \in S_i$, and the sum of all these
 363 costs is also the cardinality of our solution $\mathcal{C}$. We define the cost of $x \in S_i$
 364 as follow

$$c_x = \begin{cases} \frac{1}{\text{MINIMAL}(S_i - \mathcal{C}_{i-1})} & , \text{ if } x \in S_i \text{ has grade } k_i \text{ and } x \notin \mathcal{C}_{i-1} \\ 0 & , \text{ otherwise} \end{cases} \quad (1)$$

365 where $\mathcal{C}_{i-1} = S_1 \cup S_2 \cup \dots \cup S_{i-1}$ is the partial solution built up to $(i - 1)$ th
 366 iteration and $\text{MINIMAL}(S_i - \mathcal{C}_{i-1})$ corresponds to the number of items of grade
 367 k_i added by first time by the algorithm in the i th iteration. Seen another
 368 way, if S_i is selected, then $\text{MINIMAL}(S_i - \mathcal{C}_{i-1})$ is the number of new minimal
 369 elements that S_i contributes to the solution. Thus, we have

$$\sum_{x \in S_i} c_x = 1 \quad (2)$$

³Note that if $n' = |\mathcal{X}|/W$ is small, then the operation is made in constant time.

370 Now, for any set S belonging to the family $\mathcal{F}$ and by using our definition
 371 of cost the following relation is always true

$$\sum_{x \in S} c_x \leq \frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{|S|} = H(|S|) \quad (3)$$

where the notation $H(|S|)$ corresponds to Harmonic Number [21, 30, 35] of the integer $|S|$. In order to prove this last inequality, consider any set $S \in \mathcal{F}$ and any $i = 1, 2, \dots, |\mathcal{C}|$, and let $u_i = \text{MINIMAL}(S - (S_1 \cup S_2 \cup \dots \cup S_i))$, as the number of minimal elements in S that remain uncovered after the algorithm has selected sets $S_1, S_2, \dots, S_i$. Let $k' \geq 1$ the minimal grade for any element in the universe and $u_0 = \text{MINIMAL}(S)$ as the number of elements with grade k' in S . Let $t \leq |\mathcal{C}|$, the smallest index such that $S \subseteq (S_1 \cup \dots \cup S_{t-1} \cup S_t)$ but $\exists x \in S : x \notin (S_1 \cup \dots \cup S_{t-1})$. Thus, $u_t = 0$ and $u_j \geq u_{j+1}, \forall j = 0, 1, \dots, t-1$. Observe that $(u_{i-1} - u_i) \geq 0$ minimal elements of S are covered for the first time by S_i , for $i = 1, 2, \dots, t$. Then we have (for S)

$$\begin{aligned} \sum_{x \in S} c_x &= \sum_{i=1}^t (u_{i-1} - u_i) \cdot c_x \\ &\leq \sum_{i=1}^t (u_{i-1} - u_i) \frac{1}{\text{MINIMAL}(S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1}))} \end{aligned}$$

because it is possible that for some elements in S the cost is 0. We can also see that $\text{MINIMAL}(S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})) \geq \text{MINIMAL}(S - (S_1 \cup S_2 \cup \dots \cup S_{i-1}))$, since the algorithm selects the set S_i as the set with the largest number of uncovered elements of grade k_i (i.e. $\text{MINIMAL}(S_i) \geq \text{MINIMAL}(S), \forall S \in \mathcal{F}$ not included yet). Then, we now have

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^t (u_{i-1} - u_i) \frac{1}{\text{MINIMAL}(S - (S_1 \cup S_2 \cup \dots \cup S_{i-1}))} \\ &\leq \sum_{i=1}^t (u_{i-1} - u_i) \frac{1}{u_{i-1}} \\ &= \sum_{i=1}^t \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{u_{i-1}} \end{aligned}$$

372 giving that $u_{i-1} \geq j$, we have

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^t \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{j} \\ &= \sum_{i=1}^t \left(\sum_{j=1}^{u_{i-1}} \frac{1}{j} - \sum_{j=1}^{u_i} \frac{1}{j} \right) \end{aligned} \quad (4)$$

373 Consider now that the two internal summations at index j correspond to
374 the following Harmonic numbers

$$\sum_{j=1}^{u_{i-1}} \frac{1}{j} = \frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{u_{i-1}} = H(u_{i-1}) \quad (5)$$

375 and

$$\sum_{j=1}^{u_i} \frac{1}{j} = H(u_i) \quad (6)$$

376 Thus, replacing $H(u_{i-1})$ and $H(u_i)$ in the inner summations of inequity (4),
377 we have:

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^t (H(u_{i-1}) - H(u_i)) \\ &= H(u_0) - H(u_t) \\ &= H(\text{MINIMAL}(S)) - 0 \quad (\text{because } u_0 = \text{MINIMAL}(S)) \\ &\leq H(|S|) \quad (\text{because } \text{MINIMAL}(S) \leq |S|) \end{aligned} \quad (7)$$

378 Furthermore, given our definition of cost, we have

$$|\mathcal{C}| = \sum_{x \in \mathcal{X}} c_x \quad (8)$$

379 As each element $x \in \mathcal{X}$ is at least in one set of the optimal solution $\mathcal{C}^*$, then
380 we have

$$|\mathcal{C}| = \sum_{x \in \mathcal{X}} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \quad (9)$$

381 For inequity (3) we have

$$\begin{aligned} |\mathcal{C}| &\leq \sum_{S \in \mathcal{C}^*} |H(S)| \\ &\leq |\mathcal{C}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\}) \leq |\mathcal{C}^*| \cdot H(|\mathcal{X}|) \end{aligned} \quad (10)$$

382 On the other hand, the Harmonic Number of n , $H(n)$, is related with the
 383 natural logarithm by the following inequality [21, 30, 35].

$$H(n) = \frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{n} = \sum_{k=1}^n \frac{1}{k} \leq \ln n + 1 \quad (11)$$

Thus, replacing in inequity (10) we conclude that

$$|\mathcal{C}| \leq |\mathcal{C}^*|(\ln |\mathcal{X}| + 1)$$

384 which completes the proof that the approximation ratio of our algorithm is
 385 $\ln |\mathcal{X}| + 1$.

386 3.2. Efficient Computing Techniques in Succinct Space

387 In order to achieve sublinear time for operations on sets such as UNION,
 388 INTERSECTION and DIFFERENCE, we first represent the input datasets $(\mathcal{X}, \mathcal{F})$
 389 as a bit matrix $M_{|\mathcal{F}| \times |\mathcal{X}|}$ of $|\mathcal{F}| \cdot |\mathcal{X}|$ bits. Thus, the i th row $M[i]$ in $M_{|\mathcal{F}| \times |\mathcal{X}|}$
 390 corresponds to the representation of the i th subset S_i , and the j th element
 391 of the universe $\mathcal{X}$ is represented by the j th bit in each row of the ma-
 392 trix. We then set $M[i][j] = 1$ if the j th element is present in the subset
 393 S_i and with 0 if this element does not belong to the set. We now explain
 394 $\forall i, j, 1 \leq i \leq |\mathcal{X}|, 1 \leq j \leq |\mathcal{F}|$, how to compute $S_i \cup S_j$, $S_i \cap S_j$ or $S_i - S_j$
 395 in constant time from the bit representation.

396
 397 First, consider the singular case when $|\mathcal{X}| \leq W$, where W is the memory
 398 word size of the CPU architecture. Let p and q be two words of memory, of W
 399 bits each, to represent the $\leq W$ elements of the sets S_i and S_j respectively.
 400 Therefore, we can solve any of these set operations on p and q simply by
 401 applying the *bitwise* operators AND, OR and NEGATION. The table 1 shows
 402 how to compute these operations between sets by using bitwise operator.

403
 404 For universes of any length, $|\mathcal{X}| > W$, we can process the $|\mathcal{X}|$ bits by
 405 implement a loop routine in $O(|\mathcal{X}|/W)$ time; where p and q are bit arrays of
 406 $|\mathcal{X}|/W$ words of memory.

set operation	bitwise operation
$S_i \cup S_j$	$p \text{ OR } q$
$S_i \cap S_j$	$p \text{ AND } q$
$S_i - S_j$	$p \text{ AND } \neg q$

Table 1: The equivalent bitwise operations for set operations. The sets S_i and S_j are represented by the bit sequences p and q respectively, whose lengths are at most $|W|$ bits (i.e., $|\mathcal{X}| \leq W$ elements). The symbol $\neg$ corresponds to the negation operator (i.e., $\neg 1 = 0$ and $\neg 0 = 1$). Therefore, to compute any set operation can be done in $O(1)$ by using bitwise operator on p and q .

407 4. Experimental Results

408 In order to evaluate the empiric performance of the algorithms, we provide
409 the following implementations in C++⁴.

- 410 • **Exhaustive-SC**. It finds the optimal set-covering by means of exhaus-
411 tive searches.
- 412 • **Greedy-SC**. It is the implementation of the classical approximate greedy
413 set cover solution given by the Algorithm 1.
- 414 • **Greedy-SC- k** . It corresponds to a variant of Algorithm 1, where for
415 each iteration the two best subsets are selected to be included in the
416 solution, according to the same criteria as the classic algorithm. That
417 is, include the two sets that provide the greatest number of elements
418 not yet included in the partial solution. Given the long computation
419 time for large data sets, we try only with $k = 2$ and $k = 3$, in Figures
420 2, 3 and 4.
- 421 • **Succinct-SC**. It corresponds to our **SUCCINCT-SETCOVER** algorithm.

422 We test the implementations by using different instances $(\mathcal{X}, \mathcal{F})$ of different
423 lengths and different density for each set. All runtimes obtained are shown
424 in logarithmic scale.

⁴The repository with the code is available at https://github.com/kokee07/succinct_mscp

4.1. Setup

The experiments mentioned were performed in the Patagón Supercomputer [37], using a compute node with $2 \times$ AMD EPYC 7742 CPU (2.6Ghz, 64-cores, 256MB L3 Cache), equipped with 1TB RAM DDR4-3200Hz. Only a single thread of execution was used. The OS was Linux (Ubuntu 20.04, 64bit) running kernel 5.8.0. All programs were compiled using `g++` version 9.3.0. The runtime corresponds to real (wallclock) time from the `Chrono` library, using the `high_resolution` clock method.

4.2. Experiments with Random-Generated Datasets

In the first cycle of experiments, Figure 2, we compute the exact solution for the MSCP and the approximate responses to each instance. Since the time complexity of the optimal solution is superpolynomial, we only run experiments for instances small enough to get the answer in a reasonable amount of time. The experiment involves 4 cases with different sizes of $|\mathcal{F}|$, and each one with 5 instances with an expected universe of $|\mathcal{X}| = 70$, randomly generated using a Gaussian distribution, whose arithmetic mean was obtained from a uniform distribution over the range $[0..|\mathcal{X}|]$ and the standard deviation is $0.125|\mathcal{X}|$.

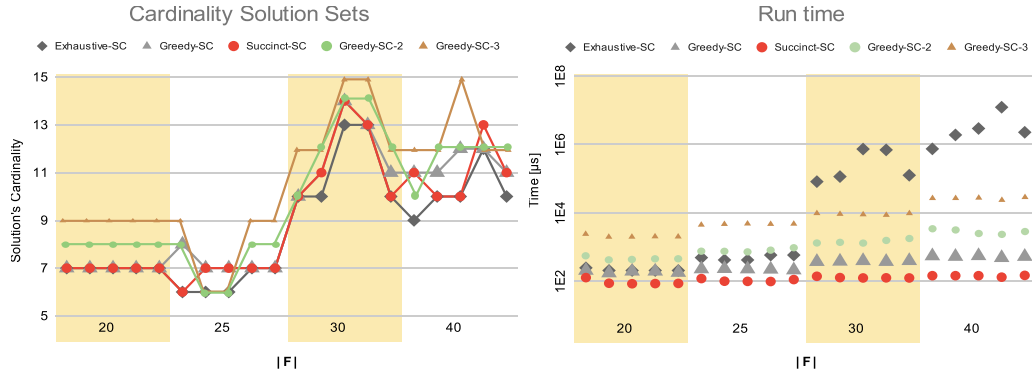


Figure 2: Cardinality (left) and running time (right) of our **Succinct-SC** approximation compared to the exact solution, **Exhaustive-SC**, the classical greedy heuristic, **Greedy-SC** and its variants **Greedy-SC- k** , with $k = 2$ and $k = 3$, for low size random generated datasets.

The dark gray line gave by the **Exhaustive-SC** corresponds to the minimum cardinality to the problem and therefore, the lower bound for any

approximation to the problem. We can appreciate how, on average, the Succinct-SC is faster (right) and smaller (left) than the Greedy-SC and the Greedy-SC- k variants.

Given the exponential growth in time required for the Exhaustive-SC to be computed on large datasets, the following experiments will only consider the approximate solutions. The second cycle of experiments, Figures 3 to 5, involves 9 random-generated cases, using the same data generator mentioned before, but increasing the value of $|\mathcal{X}|$ and $|\mathcal{F}|$ progressively. Each figure includes 3 cases with 5 instances each, grouped by the size of $|\mathcal{F}|$.

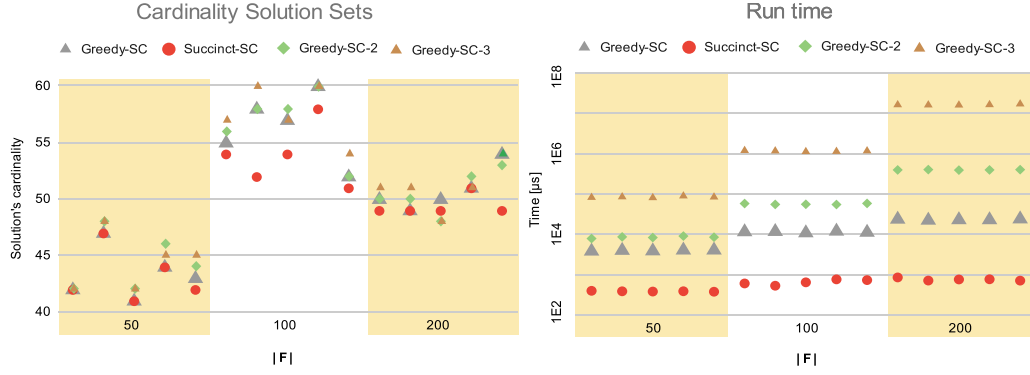


Figure 3: Cardinality and running time of our Succinct-Set Cover algorithm compared to the Greedy Set Cover for random-generated datasets with values: $|\mathcal{X}| \in [200..300]$ and $|\mathcal{F}| \in [50, 100, 200]$.

Figure 3 (left) shows an irregular difference between the quality of the solutions of both implementations, being Succinct-SC at least as good as Greedy-SC or any of the two variants Greedy-SC- k . Regarding time (right), as $|\mathcal{F}|$ increases, the distance between the execution time of each algorithm keeps increasing, at least by an order of magnitude.

Figure 4 shows the results over a medium size dataset. Achieving a more regular distance between the solutions quality, being our algorithm slightly better than the original Greedy approach and Greedy-SC-2. At the right side, each case of $|\mathcal{F}|$ size has an increase in the order of magnitude between the three solutions, evidencing the higher complexity of operations required for Greedy-SC.

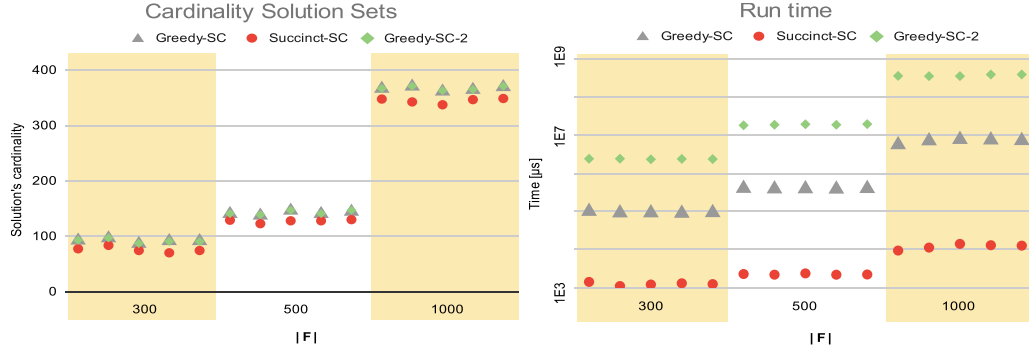


Figure 4: Cardinality and running time of our **Greedy-SC** algorithm compared to the **Greedy-SC** and **Greedy-SC-2** for random-generated datasets with values: $|\mathcal{X}_e| \in [500..2.000]$ and $|\mathcal{F}| \in [300, 500, 1.000]$.

470 The remaining experiments are with large volumes of data, where the
 471 **Greedy-SC- k** variants of the greedy approach do not run fast enough, giving
 472 unfeasible running times close to those of the optimal algorithm based on
 473 exhaustive search. Therefore, Figures 5–8 include only the **Greedy-SC** and
 474 **Succinct-SC** implementations.

475
 476 In the Figure 5, we generate higher order of $|\mathcal{F}|$. The left side shows
 477 the cardinality of the solution sets in thousands of subsets, in which our
 478 **Succinct-SC** algorithm computes a lower, and hence better, solution for
 479 each instance than the **Greedy-SC**. At the right side, the distance in order of
 480 magnitude keeps the same behaviour as the previous experiments.

481
 482 The next cycle of experiments measures the advantage of our algorithm
 483 for the case where there are elements that are included by a single subset of
 484 $\mathcal{F}$, or by very few sets of $\mathcal{F}$. In this scenario, our algorithm takes advantage
 485 by performing, in the first iteration cycles, a much more efficient pruning
 486 of the search space than the traditional greedy approach. To emulate that
 487 scenario, we implement a new generator to secure the existence of elements
 488 contained only by one subset of $|\mathcal{F}|$. To achieve this, the generator takes
 489 as input the expected universe size $|\mathcal{X}|$, the total number of subsets $|\mathcal{F}|$, a
 490 number $\mathcal{T}$ of samples to generate and a parameter p . Thus, we ensure that
 491 the first $p * |\mathcal{X}|$ elements of the universe will be randomly assigned to one set
 492 each; and then we generate all the rest of the $\mathcal{T}$ samples by using a uniform

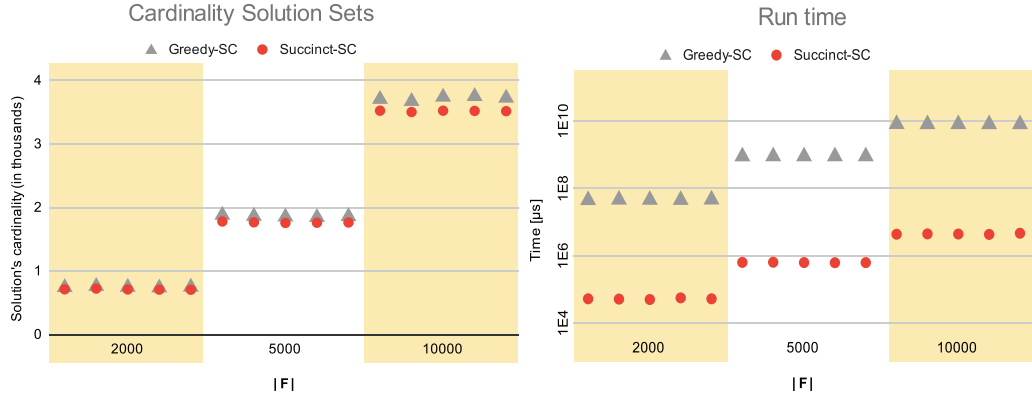


Figure 5: Cardinality and running time of our SUCCINCT-SET COVER algorithm compared to the GREEDY SET COVER for random-generated datasets with values: $|\mathcal{X}| \in [4.000..20.000]$ and $|\mathcal{F}| \in [2.000, 5.000, 10.000]$.

493 distribution over the range $[p * |\mathcal{X}| \dots |\mathcal{X}|]$.

494

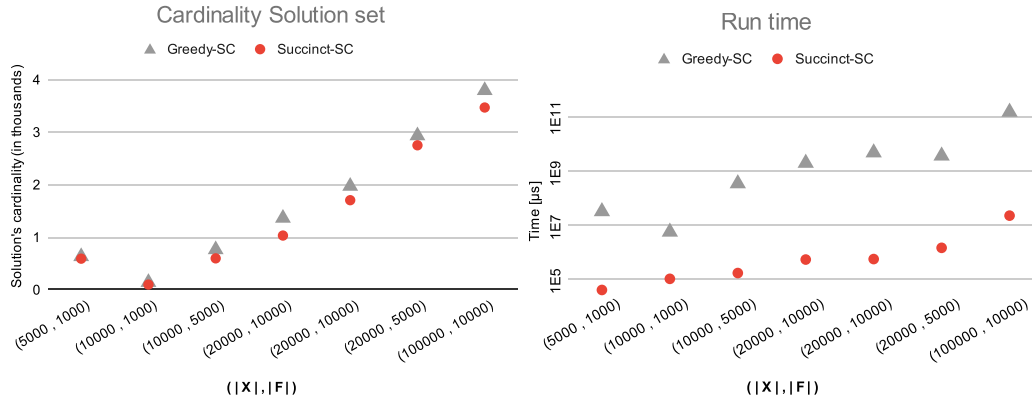


Figure 6: Cardinality and running time of our SUCCINCT-SET COVER algorithm compared to the GREEDY SET COVER for optimal random-generated datasets.

495 Figure 6 shows the results obtained from 7 different instances of optimal
 496 datasets, with the pair values $(|\mathcal{X}|, |\mathcal{F}|)$ of each case indicated in the X-
 497 axis. The results show that Succinct-SC obtains a better solution for any
 498 instance within this scenario, with a greater difference than the previous
 499 cases. Furthermore, as shown on the right side of Figure 6, in these cases
 500 a difference of up to 4 orders of magnitude is reached between the times of

501 both algorithms.

502 4.3. Experiments with Natural/Real Datasets

503 The following experiments use natural/real data sets extracted from the
 504 OR-Library [11]. In Figure 7 we use data from the SET PARTITIONING PROB-
 505 LEMS datasets, from K. L. Hoffman and D. Levine[10], used for genetic al-
 506 gorithms. In Figure 8 we test with SET COVER PROBLEM dataset, from a
 507 RAILWAY SYSTEM contributed by Paolo Nobili [9]. In Appendix Appendix
 A we detail the names of the specific sets used from these two datasets.

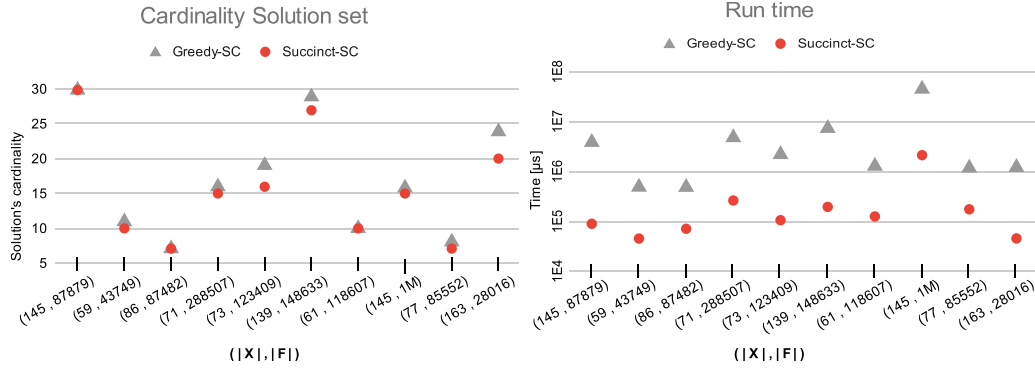


Figure 7: Cardinality and running time of our SUCCINCT-SET COVER algorithm compared to the GREEDY SET COVER for Genetic Datasets with a low $\mathcal{F}$ cardinality.

508 The decision of using dataset from genetic algorithms as a natural case
 509 is supported due to the low universe cardinality with a high number of com-
 510 binations from $\mathcal{F}$ presented. For our purpose, the constraints associated to
 511 SET PARTITIONING PROBLEMS in the dataset are ignored, such as that any
 512 element must be included in one and only one set in the solution. In Figure
 513 7 we can see, at the left, a more erratic behaviour from each case due to their
 514 natural origin. At the right, our SUCCINCT-SET COVER is at least 1 order
 515 of magnitude faster than the greedy approach.

517 As the last experiment, we test the SET COVER PROBLEM solution for
 518 a larger dataset from Italian railways [9], where a set cover solution repre-
 519 sents a selection of routes that cover all the stations. The Figure 8 shows the
 520 results from 7 cases, indicating their respective $|\mathcal{X}|$ and $|\mathcal{F}|$ at the X-axis.

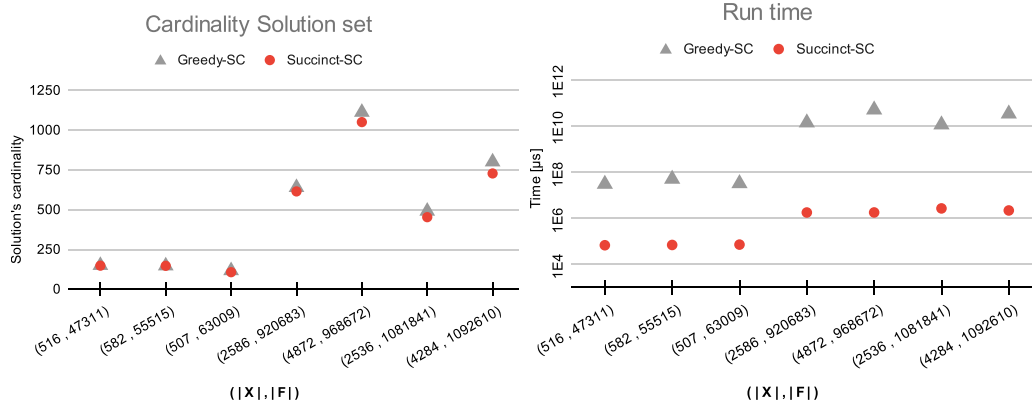


Figure 8: Cardinality and running time of our **SUCCINCT-SET COVER** algorithm compared to the **GREEDY SET COVER** for Railways Datasets.

522 The results shown in Figure 8 are consistent with all the previous
523 experiments, with our **SUCCINCT-SET COVER** algorithm achieving a solution
524 at least as good as **GREEDY SET COVER** for those cases with a low universe,
525 and a better solution for the other cases with a high universe of elements. At
526 the right side, the difference in order of magnitude between both algorithms
527 varies from 2 to 3 orders in each case. This is highly important since our
528 technique could find a better solution in minutes for any real case, in contrast
529 with the hours or days of processing attributed to the Greedy algorithm.

530 5. Conclusions and Future Work

531 We have presented a polynomial-time approximate solution to solve the
532 **MSCP**, offering an innovative algorithm that includes data compression tech-
533 niques to speed up the process. The proposed **Succinct-SC** algorithm pre-
534 processes the input dataset to create a succinct representation of the sets of
535 the family $\mathcal{F}$; which also allows to perform bit-level operations between sets
536 which is much more efficient compared to a traditional approach. Because
537 of this, we improve the execution time of the **Greedy-SC** algorithm and its
538 **Greedy-SC- k** variants significantly, while maintaining the theoretical prox-
539 imity radius unaffected. This theoretical asymptotic bound is an important
540 part of the contributions of this work. The algorithm **Succinct-SC** also offers
541 a theoretically optimal ratio approximation under the assumption that there
542 is no polynomial-time algorithm that solves any of the problems belonging
543 to the NP-Hard class. We have implemented our algorithm to measure the

544 computation time and the quality of the approximation, demonstrating that
 545 **Succinct-SC** improves the query time by orders of magnitude with respect to
 546 to the traditional greedy approximation, **Greedy-SC** algorithm, while achieving
 547 a better approximation rate in average. In summary, the results obtained
 548 with our algorithm are very favorable for the datasets tested, which included
 549 randomly generated ones as well as real ones from the state-of-the-art.

550
 551 One aspect to highlight is in relation to the efficiency achieved thanks to
 552 the succinct representation of the data, which allows computing operations
 553 between sets at high speed, always in a succinct space. We establish that
 554 the main focus when using this technique is to achieve greater speed and not
 555 necessarily compression. Since we are working on an approximation, which
 556 first has to guarantee a good quality response—we achieve this with a theo-
 557 retical proof of the bound of the algorithm—but at high speed—we achieve
 558 this by providing an implementation of a polynomial algorithm that also, in
 559 practice, solves bitwise set operations.

560
 561 The succinct/compressed representation of the data is a relevant subject
 562 for future work, where one goal can be to design an optimal algorithm that
 563 works in a succinct space, also incorporating CPU/GPU parallelism. In terms
 564 of compression and based on the tested scenarios, we expect that by using
 565 the proposed succinct representation it could be possible to: *i*) build new
 566 implementations to solve MSCP optimally, processing large volumes of data
 567 that is not possible today using classical computing; *ii*) speedup the current
 568 algorithms that find the optimal solution for the MSCP, and also that this
 569 technique will be useful for others NP problems.

570 **6. Acknowledgment**

571 This research was supported by the Fondecyt grant #11221029 and by
 572 the Patagón supercomputer of Universidad Austral de Chile (FONDEQUIP
 573 EQM180042).

574 **References**

- 575 [1] Abrache, J., Crainic, T. G., Gendreau, M., and Rekik, M. (2007). Com-
 576 binatorial auctions. *Annals OR*, 153:131–164.

- 577 [2] Balas, E. and Carrera, M. C. (1996). A dynamic subgradient-based
578 branch-and-bound procedure for set covering. *Oper. Res.*, 44(6):875–890.
- 579 [3] Balas, E. and Ho, A. (1980). *Set covering algorithms using cutting planes,*
580 *heuristics, and subgradient optimization: A computational study*, pages 37–
581 60. Springer Berlin Heidelberg, Berlin, Heidelberg.
- 582 [4] Balas, E. and Ho, A. (2009). *Set covering algorithms using cutting*
583 *planes, heuristics, and subgradient optimization: A computational study*,
584 volume 12, pages 37–60.
- 585 [5] Bar-Yehuda, R. and Even, S. (1981). A linear-time approximation al-
586 gorithm for the weighted vertex cover problem. *Journal of Algorithms*,
587 2(2):198–203.
- 588 [6] Barnhart, C., Cohn, A. M., Johnson, E. L., Klabjan, D., Nemhauser,
589 G. L., and Vance, P. H. (2003). *Airline Crew Scheduling*, pages 517–560.
590 Springer US, Boston, MA.
- 591 [7] Beasley, J. (1987). An algorithm for set covering problem. *European*
592 *Journal of Operational Research*, 31(1):85–93.
- 593 [8] Beasley, J. and Jørnsten, K. (1992). Enhancing an algorithm for set
594 covering problems. *European Journal of Operational Research*, 58(2):293–
595 300. Practical Combinatorial Optimization.
- 596 [9] Beasley, J. E. Or-library, set cover problems dataset. [dataset].
- 597 [10] Beasley, J. E. Or-library, set partitioning problems dataset. [dataset].
- 598 [11] Beasley, J. E. (1990). Or-library: Distributing test problems by elec-
599 tronic mail. *Journal of the Operational Research Society*, 41(11):1069–
600 1072.
- 601 [12] Bilal, N., Galinier, P., and Guibault, F. (2013). A new formulation of
602 the set covering problem for metaheuristic approaches. *ISRN Operations*
603 *Research*, 2013:10 pages.
- 604 [13] Burhan, H. M., Attea, B. A., Abboud, A. D., Abbas, M. N., and Al-
605 Ani, M. (2021). Evolutionary multi-objective set cover problem for task
606 allocation in the internet of things. *Applied Soft Computing*, 102:107097.

- 607 [14] Cacchiani, V., Hemmelmayr, V., and Tricoire, F. (2014). A set-covering
608 based heuristic algorithm for the periodic vehicle routing problem. *Discrete*
609 *Applied Mathematics*, 163:53–64. Matheuristics 2010.
- 610 [15] Caprara, A., Fischetti, M., and Toth, P. (1999). A heuristic method for
611 the set covering problem. *Operations Research*, 47(5):730–743.
- 612 [16] Caprara, A., Fischetti, M., Toth, P., Vigo, D., and Guida, P. (2000).
613 Algorithms for railway crew management. *Mathematical Programming*,
614 79.
- 615 [17] Chan, T. M., Grant, E., Könemann, J., and Sharpe, M. (2012).
616 Weighted capacitated, priority, and geometric set cover via improved quasi-
617 uniform sampling. In *Proceedings of the Twenty-Third Annual ACM-SIAM*
618 *Symposium on Discrete Algorithms*, SODA ’12, page 1576–1585, USA. So-
619 ciety for Industrial and Applied Mathematics.
- 620 [18] Chen, J., Mi, J., and Lin, Y. (2020). A graph approach for fuzzy-rough
621 feature selection. *Fuzzy Sets and Systems*, 391:96–116. Computer Science.
- 622 [19] Chvatal, V. (1979). A greedy heuristic for the set-covering problem.
623 *Math. Oper. Res.*, 4(3):233–235.
- 624 [20] Contardo, C. and Hertz, A. (2021). An exact algorithm for a class of
625 geometric set-cover problems. *Discrete Applied Mathematics*, 300:25–35.
- 626 [21] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009).
627 *Introduction to Algorithms, 3rd Edition*. MIT Press.
- 628 [22] Feige, U. (1998). A threshold of $\ln n$ for approximating set cover. *J.*
629 *ACM*, 45(4):634–652.
- 630 [23] Fisher, M. L. and Kedia, P. (1990). Optimal solution of set
631 covering/partitioning problems using dual heuristics. *Manage. Sci.*,
632 36(6):674–688.
- 633 [24] Foster, B. A. and Ryan, D. M. (1976). An integer programming approach
634 to the vehicle scheduling problem. *Journal of the Operational Research*
635 *Society*, 27(2):367–384.

- [25] Gomes, F. C., Meneses, C. N., Pardalos, P. M., and Viana, G. V. R. (2006). Experimental analysis of approximation algorithms for the vertex cover and set covering problems. *Computers & Operations Research*, 33(12):3520–3534. Part Special Issue: Recent Algorithmic Advances for Arc Routing Problems.
- [26] Hochbaum, D. S. (1982). Approximation algorithms for the set covering and vertex cover problems. *SIAM Journal on computing*, 11(3):555–556.
- [27] Johnson, D. S. (1973). Approximation algorithms for combinatorial problems. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, STOC '73, page 38–49, New York, NY, USA. Association for Computing Machinery.
- [28] Karp, R. M. (1972). *Reducibility among Combinatorial Problems*, pages 85–103. Springer US, Boston, MA.
- [29] Knuth, D. E. (1973). *The Art of Computer Programming. Vol.3: Sorting and Searching*.
- [30] Knuth, D. E. (1981). *The Art of Computer Programming*, volume 2: Seminumerical Algorithms. Addison-Wesley.
- [31] Lan, G., DePuy, G. W., and Whitehouse, G. E. (2007). An effective and simple heuristic for the set covering problem. *European Journal of Operational Research*, 176(3):1387–1403.
- [32] Lim, C. L., Moffat, A., and Wirth, A. (2014). Lazy and eager approaches for the set cover problem. In *Proceedings of the Thirty-Seventh Australasian Computer Science Conference - Volume 147*, ACSC '14, page 19–27, AUS. Australian Computer Society, Inc.
- [33] Lund, C. and Yannakakis, M. (1994). On the hardness of approximating minimization problems. *J. ACM*, 41(5):960–981.
- [34] Musser, D. R. (1997). Introspective sorting and selection algorithms. *Softw. Pract. Exper.*, 27(8):983–993.
- [35] Olaikhan, A. (2021). *An Introduction to the Harmonic Series and Logarithmic Integrals: For High School Students Up to Researchers*. Amazon Digital Services LLC - KDP Print US.

- 667 [36] Owais, M., Osman, M. K., and Moussa, G. (2016). Multi-objective
668 transit route network design as set covering problem. *IEEE Transactions*
669 *on Intelligent Transportation Systems*, 17(3):670–679.
- 670 [37] Patagón Supercomputer (2021). <https://patagon.uach.cl>. Accessed
671 January 21, 2023.
- 672 [38] Peleg, D., Schechtman, G., and Wool, A. (1997). Randomized approxi-
673 mation of bounded multicovering problems. *Algorithmica*, 18(1):44–66.
- 674 [39] Peuzin-Jubert, M., Polette, A., Nozais, D., Mari, J.-L., and Pernot, J.-
675 P. (2021). Survey on the view planning problem for reverse engineering
676 and automated control applications. *Computer-Aided Design*, 141:103094.
- 677 [40] Ribeiro, C. C., Minoux, M., and Penna, M. C. (1989). An optimal
678 column-generation-with-ranking algorithm for very large scale set parti-
679 tioning problems in traffic assignment. *European Journal of Operational*
680 *Research*, 41(2):232–239.
- 681 [41] Rubin, J. (1973). A technique for the solution of massive set cover-
682 ing problems, with application to airline crew scheduling. *Transportation*
683 *Science*, 7(1):34–48.
- 684 [42] Spensieri, D., Åblad, E., Bohlin, R., Carlson, J. S., and Söderberg, R.
685 (2021). Modeling and optimization of implementation aspects in indus-
686 trial robot coordination. *Robotics and Computer-Integrated Manufactur-*
687 *ing*, 69:102097.
- 688 [43] Vemuganti, R. R. (1998). *Applications of Set Covering, Set Packing and*
689 *Set Partitioning Models: A Survey*, pages 573–746. Springer US, Boston,
690 MA.
- 691 [44] Wang, X. and Ju, S. (2009). A set-covering-based approach for over-
692 lapping resource selection in distributed information retrieval. In *2009*
693 *WRI World Congress on Computer Science and Information Engineering*,
694 volume 4, pages 272–276.

695 Appendix A. Dataset Details

696 In this appendix we detail the names of the sets used from the OR-Library
697 [11]. Specifically: SET PARTITIONING PROBLEMS dataset included by K. L.
698 Hoffman and D. Levine[10] (with the first 10 rows arranged from left to right
699 in Figure 7) and RAILWAY SYSTEM by Paolo Nobili [9] (with the last 7 rows
700 arranged from left to right in Figure 8).

article dataset	set's name	rows $ \mathcal{X} $	columns $ \mathcal{F} $
[10]	sppnw02	145	87879
[10]	sppnw03	50	43749
[10]	sppnw04	86	87482
[10]	sppnw05	71	288507
[10]	sppnw14	73	123409
[10]	sppnw16	139	148633
[10]	sppnw17	61	118607
[10]	sppus01	145	1000000
[10]	sppus03	77	85552
[10]	sppus04	163	28016
[9]	rail516	516	47311
[9]	rail582	582	55515
[9]	rail507	507	63009
[9]	rail2536	2536	1081841
[9]	rail2586	2586	920683
[9]	rail4284	4284	1092610
[9]	rail4872	4872	968672

Table A.2: Names of the sets used in the experiments with natural/real dataset.